

GateCheck: Executable Semantic Contracts for Silent Failures in Learned Gates and Attention Components

ANONYMOUS AUTHOR(S)

Neural attention and gating code can run, train, and preserve tensor shapes while silently ignoring a mask, option, branch, or gradient path. End-to-end accuracy cannot establish that a named component implements its claimed mechanism. We introduce GATECHECK, a lightweight suite of executable semantic contracts covering shape, finiteness, non-degeneracy, input influence, forward effect, parameter-gradient connectivity, batch independence, protected-input invariance, and declared-option influence. GATEBENCH-v2 combines spatial and sequence gates: 63 semantic-mutant configurations and 33 valid controls receive 17,280 executions over seeds, sizes, batches, and input distributions. The full local suite detects all controlled mutants and flags no controls (mutant recall 1.00, exact 95% interval 0.943–1.00; control specificity 1.00, 0.894–1.00), whereas runtime/shape testing detects 9.5%. A framework-parity experiment obtains identical decisions for five definitions in PyTorch and JAX. A deliberately small public-patch pilot adds five buggy/fixed pairs derived from immutable Transformers and Diffusers commits: every buggy replay violates its prespecified contract and every fixed replay passes, but the interval is wide (0.478–1.00) and the replays are not full historical environments. Finally, a patient-grouped brain-tumor case study proves that a singleton-softmax gate is an identity with zero parameter gradient; two repairs restore mechanism but do not establish a clean Dice benefit on 35 held-out patients. GATECHECK is a pre-training integrity oracle, not a guarantee of model quality or universal bug detection.

CCS Concepts: • **Software and its engineering** → **Software testing and debugging**; • **Computing methodologies** → *Machine learning*.

Additional Key Words and Phrases: deep-learning software, property-based testing, metamorphic testing, attention, gates, silent bugs

ACM Reference Format:

Anonymous Author(s). 2027. GateCheck: Executable Semantic Contracts for Silent Failures in Learned Gates and Attention Components. In *Proceedings of The ACM International Conference on the Foundations of Software Engineering (FSE 2027)*. ACM, New York, NY, USA, 14 pages.

1 Introduction

Neural networks are programs as well as statistical models. A training run can finish, its loss can fall, and its predictions can look plausible even when a named submodule does not implement the mechanism described in a paper. This class of failure is particularly dangerous for learned multiplicative gates, including channel and spatial attention: tensor broadcasting makes many unintended expressions legal, and a residual backbone can learn around an inactive branch.

The motivating defect was discovered while auditing a brain-tumor segmentation project. Global pooling and a one-output convolution produced a tensor of shape $B \times 1 \times 1 \times 1$, followed by softmax over its single channel. Every attention value was exactly one, so the module returned its input and its score parameters received zero gradient. No exception was raised. A conventional smoke test, parameter count, training curve, or final Dice score could not establish that the purported attention mechanism was inactive.

Empirical studies confirm that faults in deep-learning software are varied and often silent. A taxonomy derived from 1,059 artifacts and developer studies includes data, model, training, API, and tensor faults [13]. A Keras/TensorFlow study reproduced 77 silent bugs among 1,168 collected

issues [31]. Defects4ML provides 100 reproducible ML-system bugs [22]; a later audit of five benchmarks found that only 18.5% of 314 eligible faults met all realism conditions and only 52% of reproduction attempts succeeded [16]. Thus, a public “real-fault” label does not by itself establish faithful execution. Existing tools address neighboring problems: differential behavior [26], rare inputs [23], cross-backend inconsistencies [27], compiler defects [18], shape errors [17], numerical failures [34, 35], mutation adequacy [14, 19], and validity-constrained generation [28]. None substitutes for a declared local contract stating what one component must influence or preserve.

This paper asks four research questions:

- **RQ1:** Which executable properties distinguish functioning gates from silent semantic defects?
- **RQ2:** How much defect coverage is lost when testing stops at runtime, shapes, or finite values?
- **RQ3:** Do the same contract ideas expose patch-derived public defects and remain stable across frameworks?
- **RQ4:** What does component integrity change—and not change—in an end-to-end medical-imaging case study?

We make four contributions. First, we formalize seven inexpensive local gate contracts and two declarative influence/invariance schemas. Second, GATEBENCH-v2 instantiates 21 mutation mechanisms and 11 valid spatial/sequence families at three widths: 63 mutants, 33 controls, and a prespecified 17,280-execution stress matrix. Third, we add reproducible evidence outside that controlled benchmark: PyTorch–JAX decision parity and five provenance-linked public-patch micro-replays. Fourth, we connect software integrity to a patient-grouped segmentation study and show how slice pseudoreplication can compound a software error with an invalid statistical conclusion. The artifact includes source, tests, every execution row, protocols, analyses, and manuscript source.

The claims are deliberately bounded. Perfect detection on author-designed cases is not evidence of universal detection. The five public patches are a convenience pilot and are reduced semantic replays, not a representative corpus. No practitioner study or independent contract annotation has been completed. These boundaries are central to interpreting the results.

2 Background and Related Work

2.1 Learned gates and semantic failure

Squeeze-and-Excitation (SE) maps globally pooled features to independent sigmoid channel weights [7]; CBAM combines channel and spatial gates [33]; Attention U-Net conditions skip features on decoder context [24]. These mechanisms impose different contracts. A sigmoid channel gate permits independent weights, whereas a channel-softmax gate imposes competition. A spatial softmax is meaningful only when its normalized spatial support contains more than one element. Axis and cardinality are therefore part of the semantics, not merely implementation details.

The forward form studied here is

$$G_{\theta}(X) = X \odot A_{\theta}(X), \quad (1)$$

where X is either spatial ($B \times C \times H \times W$) or sequential ($B \times T \times D$), A_{θ} may broadcast over documented axes, and θ denotes trainable gate parameters. Residual variants replace Eq. (1) with $(1 - \rho)X + \rho X \odot A_{\theta}(X)$ for declared $\rho > 0$. A syntactically valid gate is not necessarily functioning: $A \equiv 1$, $\rho = 0$, a detached A , or computed-but-ignored A can all preserve valid output shapes.

PROPOSITION 1 (SINGLETON-SOFTMAX COLLAPSE). *Let $z \in \mathbb{R}^1$ and $a = \text{softmax}(z)$. Then $a = 1$, $\partial a / \partial z = 0$, and the multiplicative gate $G(X) = X \odot a$ is the identity for every X .*

PROOF. $a_1 = e^{z_1} / \sum_{j=1}^1 e^{z_j} = 1$. The softmax Jacobian is $\partial a_i / \partial z_j = a_i(\delta_{ij} - a_j)$, so $\partial a_1 / \partial z_1 = 1(1 - 1) = 0$. Substitution into Eq. (1) gives $G(X) = X$ and the chain rule gives zero gradient for all score-layer parameters. $\square$

The same argument holds whenever softmax normalizes an axis of length one. It does not imply that softmax attention is generally invalid; it identifies an axis-cardinality incompatibility.

2.2 Testing deep-learning software

DeepXplore and TensorFuzz search for behavioral errors in complete trained models [23, 26]. CRA-DLE compares backends, while NNSmith generates valid graphs and uses differential testing to expose compiler defects [18, 27]. PyTea statically collects tensor-shape constraints and can identify incompatible operations before training [17]. These approaches are complementary to local semantic contracts: a gate can be shape-correct and backend-consistent while remaining a deterministic no-op.

Property-driven work is closest in spirit. QuickCheck established the executable-property pattern for generated tests [4]. MLCheck specifies model-level requirements for fairness, monotonicity, and security [30]. aNNoTest uses annotations and property-based generation to produce valid inputs for neural-network programs; it exposed 94 bugs in 19 programs [28]. GATECHECK narrows the primary boundary to a learned component and combines forward-effect, backward-connectivity, counterfactual independence, and declared influence/preservation contracts.

Mutation testing evaluates whether tests detect injected faults. DeepMutation defines source- and model-level operators, and DeepCrime grounds pre-training operators in observed fault classes [14, 19]. Our mutations do not retrain models or evaluate dataset adequacy. They perturb component semantics, and a mutant is “killed” when at least one declared gate contract fails. This scope difference matters: GATEBENCH-v2 measures the present suite against controlled mechanisms, not the frequency or representativeness of real defects.

3 GATECHECK

3.1 Contract probes

Let X, X' be distinct inputs; let $\tilde{X}$ change only sample 1 in a batch; and let $D(U, V) = \|U - V\|_2 / (\|V\|_2 + 10^{-12})$. Seven local probes apply directly to multiplicative gates:

- (1) **Runtime/shape.** Calls to A_θ and G_θ must succeed; $G_\theta(X)$ must match X ; attention must preserve the batch axis and obey declared broadcasting.
- (2) **Finiteness.** $A_\theta(X)$ and $G_\theta(X)$ contain neither NaN nor infinity.
- (3) **Non-constancy.** $\text{std}(A_\theta(X)) \geq \tau_c$.
- (4) **Input response.** $D(A_\theta(X'), A_\theta(X)) \geq \tau_r$.
- (5) **Forward effect.** $D(G_\theta(X), X) \geq \tau_f$ for a module declared to modulate its input.
- (6) **Gradient connectivity.** For probe loss $L = \text{mean}(G_\theta(X)^2)$ and a trainable gate, $\sum_{p \in \theta} \|\nabla_p L\|_1 \geq \tau_g$.
- (7) **Batch independence.** Changing sample 1 must not change samples 2, $\dots$, B . Define

$$q_b = \max \left\{ \frac{D(G(\tilde{X})_{2:B}, G(X)_{2:B})}{D(A(X)_{2:B}, A(X)_{2:B})} \right\}; \quad q_b \leq \tau_b. \quad (2)$$

The prespecified thresholds are $\tau_c = \tau_r = \tau_f = 10^{-6}$, $\tau_g = 10^{-10}$, and $\tau_b = 10^{-5}$. They are numerical guardrails rather than hypothesis tests. A legitimate design may opt out of an inapplicable contract: an intentionally constant initialization can temporarily violate non-constancy; batch-normalization in training mode intentionally couples samples; a residual gate with a declared zero

mixture intentionally begins as an identity. The executable specification must therefore accompany the architectural claim.

Two additional schemas cover attention interfaces. A *declared-option influence* contract supplies option values $o \neq o'$ and requires $D(F(X; o), F(X; o')) \geq \tau_r$ when the option is documented to affect execution. A *protected-input invariance* contract changes only a region R excluded by mask M and requires $D(F(X, M), F(X_{R \leftarrow X'}, M)) \leq \tau_b$. These schemas test, for example, whether attention scaling, conditioning, processor arguments, or padding masks reach the intended computation. They require a domain assertion about what should change or remain invariant; GATECHECK does not infer intent from a function name.

3.2 Counterfactual construction

For each run, parameters and inputs are seeded. For spatial tensors, X' reverses the final spatial axis and applies an affine transformation; for sequences it reverses the token axis and applies the same transform. $\tilde{X}$ changes only sample 1. These changes are large enough to avoid equating floating-point noise with semantic response. Gate parameters use framework defaults except in deliberately saturated mutants. The local suite performs one forward/backward pass and two no-gradient counterfactual passes. Influence and invariance schemas execute their declared paired scenarios.

3.3 Decision rule

An execution is flagged if any applicable contract fails. A configuration is classified as faulty when at least half of its stress executions are flagged. The majority aggregation guards against an isolated numerical anomaly, although every mutant and control in our experiment has a detection rate of exactly one or zero, respectively. We report case-level rates because the 180 stress executions for one configuration are repeated measurements, not independent defect cases.

3.4 Generative-AI assistance

The author used OpenAI ChatGPT/Codex during the research lifecycle to assist literature discovery, propose and revise experimental code, run scripted analyses, and draft manuscript text. All reported programs were executed in the disclosed environment; numerical claims are generated from archived machine-readable outputs; public-patch provenance is anchored to immutable commit hashes. No AI system was treated as an independent annotator or human participant. The benchmark is labeled author-designed, and the author remains accountable for the method, code, citations, analysis, and claims. This disclosure is included because the venue requires detailed reporting when generative AI affects research design, experiments, code, or analysis.

4 Study Design

4.1 GATEBENCH-v2 controlled cases

The spatial partition retains six gate families at channel widths $C \in \{4, 8, 16\}$: sigmoid channel, sigmoid spatial, channel softmax over C , spatial softmax over HW , CBAM-like channel-spatial, and residual sigmoid. Its 33 mutants instantiate 11 operators:

- (1) softmax over a singleton channel;
- (2) softmax over a singleton spatial support;
- (3) detached attention tensor;
- (4) attention computed but ignored;
- (5) residual mixture multiplied by zero;
- (6) softmax over the batch axis;

Table 1. Frozen public-patch pilot. Hashes link the replay to developer evidence; “contract” is selected before replay outcomes.

Project / PR / commit	Developer-described defect		Contract
Transformers #47946 / e0aeab4c9f71	encoder mask overwritten/discarded		protected-region invariance
Transformers #44397 / 8757098be4d2	scaling ignored in SDPA/FlashAttention		option influence
Diffusers #9517 / acd6d2c42f0f	attention kwargs not forwarded		argument influence
Transformers #17437 / d444edb3f63d	fp16 masked softmax NaNs		finiteness
Transformers #48679 / 5bed62964cfe	first-step conditioning dropped		conditioning preservation

- (7) first sample broadcast to the batch;
- (8) sigmoid saturated near one;
- (9) sigmoid saturated near zero;
- (10) division by zero producing non-finite values; and
- (11) an incompatible attention shape.

The sequence partition instantiates five valid families—token sigmoid, feature sigmoid, feature softmax, residual token, and recurrent-update gates—at feature widths $D \in \{4, 8, 16\}$. Ten sequence mutations cover singleton feature softmax, detachment, ignored and zero-mixture paths, batch-axis softmax, first-sample broadcast, positive and negative saturation, non-finite arithmetic, and incompatible shape. This contributes 30 mutants and 15 controls. Some mechanisms intentionally overlap the spatial partition: replication across tensor organizations tests whether the contracts depend on one layout.

Each configuration receives five seeds {17, 43, 101, 2027, 4099}, three sizes, batches {2, 4, 7}, and Gaussian, uniform, ReLU-sparse, and positive-skew inputs, totaling 180 executions. The spatial and sequence partitions contain 96 configurations (63 mutants and 33 controls) and 17,280 executions. The factorial matrix and thresholds were fixed before the final reported run. Controlled experiments use float32 PyTorch 2.9.1+cpu.

4.2 Public-patch pilot

We queried the GitHub commit-search API on 13 September 2026 for attention fixes in Transformers, Diffusers, pytorch-image-models, and MONAI. Before executing outcomes, a candidate required an immutable public patch, an explicit developer description or regression test, a semantic mapping to a declared contract, and a CPU-executable reduction without model weights or proprietary data. Five pairs met these criteria; a sixth XGLM fp16 patch was excluded as a duplicate of the OPT mechanism. Table 1 gives the frozen set. The complete queries, hashes, and exclusion rule are archived.

Each pair is a minimal semantic replay of the changed behavior, not a checkout of the full historical dependency stack. We run buggy and fixed forms on the same generated inputs for five seeds where randomness is relevant. A pair counts as detected only if all buggy executions fail and all fixed executions pass. This pilot can test mechanism alignment; it cannot estimate real-world prevalence or generalization.

The corresponding public commits are cited as developer evidence [8–12]. Because selection is small and purposive, we do not compare against the scale or representativeness of Defects4ML.

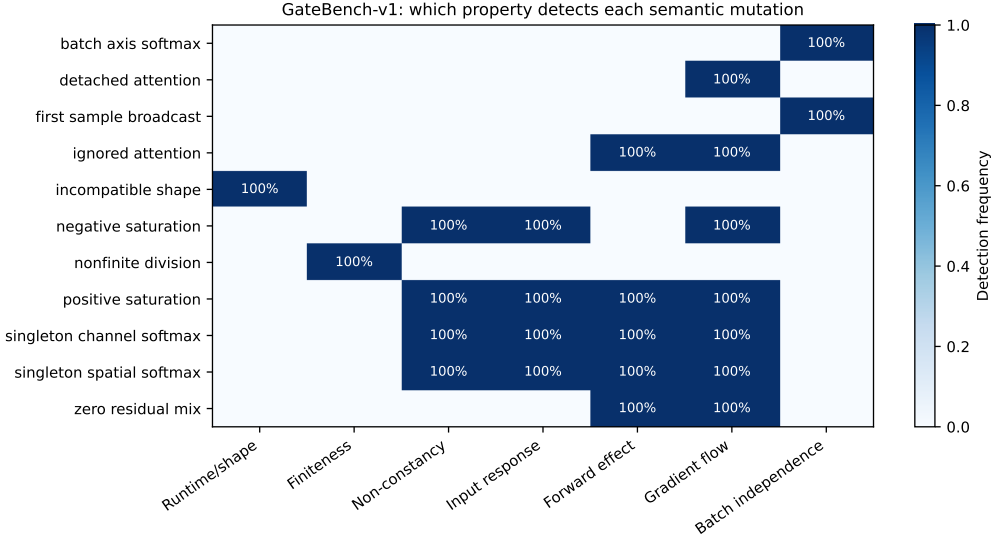


Fig. 1. Spatial-partition probe activation frequency for each mutation across 540 executions (three widths, five seeds, three shapes, three batches, four distributions). Blank cells are zero. Several defects require forward/backward or batch counterfactuals.

4.3 Cross-framework parity

We implement five matched definitions in PyTorch 2.9.1+cpu and JAX 0.11.1 [1, 25]: one valid sigmoid gate and four faults (singleton softmax, ignored forward path, batch-axis softmax, and positive saturation). Five seeds yield 50 executions. The outcome is whether the two independent implementations make the same pass/fail decision, not numerical tensor equality.

4.4 Metrics and uncertainty

Primary metrics are configuration-level mutant recall and control specificity. We include two-sided 95% Clopper–Pearson intervals to communicate the uncertainty that remains despite observed perfect separation. Secondary analyses report probe activation, cumulative probe-family ablations, a joint threshold multiplier sweep from 10^{-4} to 10^4 , cross-framework agreement, and wall-clock time. Controlled cases are deterministic given a seed; intervals summarize finite-set discrimination, not defect prevalence. The public-patch pilot receives a separate interval because combining it with synthetic mutants would obscure its much smaller independent evidence base.

5 Benchmark Results

5.1 RQ1: detection and specificity

Across both partitions, the full suite detects all 63 mutant configurations and flags none of the 33 controls: recall 1.00 (exact 95% CI 0.943–1.00) and specificity 1.00 (0.894–1.00). Separately, the spatial result is 33/33 mutants and 0/18 controls; the sequence result is 30/30 and 0/15. At execution level, every mutant run is flagged and every control run passes, but these repetitions are not independent defects.

Figure 1 shows mechanistic complementarity. Shape and non-finite mutants have direct signatures. Singleton softmax and positive saturation jointly fail value dynamics, forward effect, and

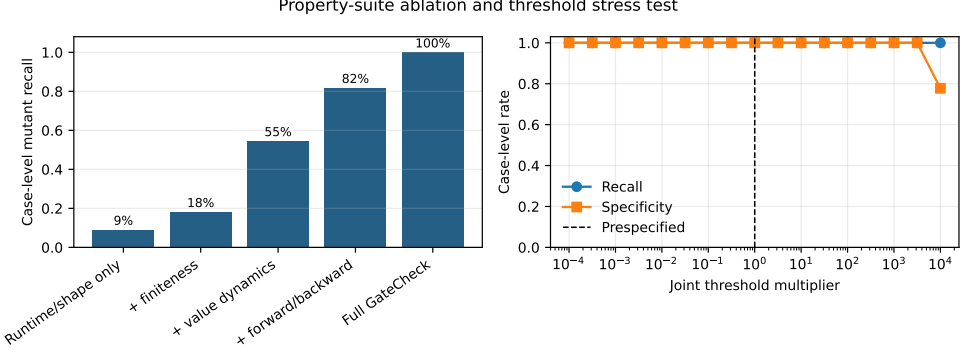


Fig. 2. Left: cumulative property-family ablation. Right: joint multiplication of all numerical thresholds. Perfect separation persists through a 3,162 $\times$ multiplier; at 10^4 , four valid controls are falsely flagged.

gradient flow. Negative saturation is nonconstant only below numerical tolerance and is input-insensitive and gradient-isolated, but it does change the forward output toward zero. Detached attention remains input-responsive and affects the forward pass; only the gradient probe exposes its disconnected parameters. Ignored attention and zero mixture are caught by forward effect and gradient connectivity. Batch-axis softmax and first-sample broadcast pass all per-sample value and gradient probes; only the counterfactual batch test detects them.

5.2 RQ2: ablations and thresholds

Across the 63 controlled mutants, runtime/shape testing detects 6 (9.5% case-level recall). Adding finiteness reaches 12/63 (19.0%). Adding non-constancy and input response reaches 33/63 (52.4%); forward effect and gradient flow reach 51/63 (81.0%). Only batch independence reaches 63/63. All cumulative variants retain 33/33 control specificity. Figure 2 visualizes the spatial partition; the archived combined table contains the values above.

Leave-one-probe-out analysis gives a second view. Omitting runtime/shape, finiteness, or gradient connectivity reduces recall to 90.9%; omitting batch independence reduces it to 81.8%. Omitting non-constancy, input response, or forward effect alone leaves recall at 100% because other probes redundantly expose the corresponding mutants. Redundancy is desirable for diagnosis: a constant map, an identity forward path, and zero gradients distinguish different consequences of the same singleton-softmax root cause.

The threshold sweep preserves recall and specificity from 10^{-4} through 3,162 times the defaults. At 10^4 , specificity decreases to 0.778 while recall remains one. This broad plateau shows that the present cases are not separated by a finely chosen decimal threshold. It does not calibrate thresholds for other architectures or lower-precision execution.

5.3 Cost

Median end-to-end probe time is 2.20 ms for a spatial execution (95th percentile 6.94 ms) and 0.72 ms for a sequence execution (1.35 ms) on the available CPU host. Timing includes module construction and backward probing. Hardware values are descriptive, not cross-platform performance claims.

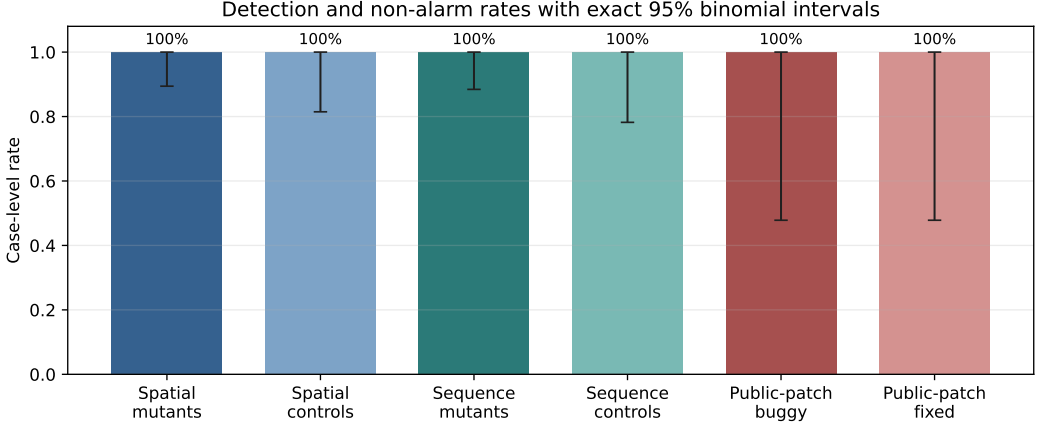


Fig. 3. Case-level detection/non-alarm rates with exact binomial intervals. Perfect point estimates have different evidential strength: the five public-patch pairs have a substantially wider interval than the controlled partitions.

6 RQ3: Public Patches and Framework Parity

All five patch-derived buggy forms violate their prespecified contract for every applicable seed, while all five fixed forms pass. Case-level buggy recall is 1.00 and fixed specificity is 1.00; both exact 95% intervals are 0.478–1.00. The wide lower bound is more informative than the point estimate: five hand-screened pairs cannot support broad reliability claims.

The defects are shape-correct except for none—indeed, ordinary runtime/shape checks detect 0/5. Finiteness detects only the OPT fp16 failure (1/5). The remaining pairs require an influence, preservation, or invariance oracle. The GPT-2 mask replay changes masked encoder states to 10^6 ; the buggy output changes by a median relative L_2 distance of 1.36×10^6 , whereas the fixed output is invariant. The scaling and processor-argument bugs show exactly zero response to their declared options before repair and nonzero response after repair. The MusicGen replay drops conditioning when an empty pre-created cache is present; the fixed guard preserves it at the first step.

The cross-framework experiment yields identical pass/fail decisions for all 25 case–seed pairs in PyTorch and JAX. Both identify all four fault definitions and pass the valid control. This result shows that the numerical contracts need not depend on autograd syntax in one framework; it does not constitute broad TensorFlow, accelerator, or compiler validation.

7 RQ4: Brain-Tumor Case Study

7.1 Protocol

We revisit the public Cheng contrast-enhanced T1 MRI dataset [2, 3]: 3,064 slices from 233 patients. Patient identifiers are partitioned before preprocessing into 163 training patients (2,082 slices), 35 validation patients (443 slices), and 35 test patients (539 slices), stratified by tumor type. The test set is untouched by checkpoint selection or design choices.

All variants share a compact three-level 2D U-Net [29] with widths 4, 8, 16, and bottleneck 32. Images are robust-percentile normalized and resized to 96×96 ; masks use nearest-neighbor resizing. We compare plain U-Net, a direct sigmoid-channel repair, and a residual channel–spatial gate. Each model is trained for 18 epochs with BCE plus soft Dice loss, AdamW, and three matched seeds (17, 43, 101). Evaluation includes clean images, additive Gaussian noise ($\sigma = 0.08$), and a

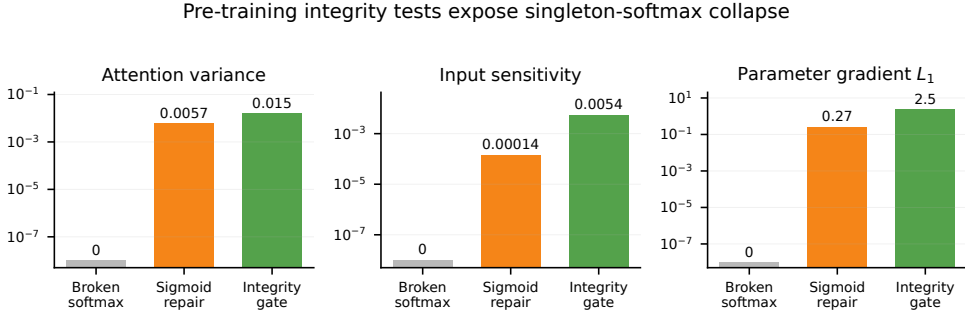


Fig. 4. Fixed-input audit of the motivating singleton-softmax gate and two functional repairs. Zeros are plotted at 10^{-8} only for visibility on logarithmic axes. Absolute gradient magnitudes are pass/fail evidence, not a ranking across parameterizations.

Table 2. Patient-level Dice, averaged across three seeds ($n = 35$ patients). Parentheses give the clean-condition 95% patient-bootstrap interval.

Model	Clean	Noise	Bias field
Plain U-Net	.559 (.480–.632)	.522	.534
Sigmoid repair	.572 (.493–.650)	.538	.546
Integrity gate	.569 (.488–.641)	.536	.548

diagonal multiplicative bias field ranging 0.65–1.35. These are generic stress tests, not scanner-physics models.

The scientific unit is the patient. Slice metrics are averaged within patient, then across seeds. Confidence intervals use 5,000 patient bootstrap samples [5]; two-sided paired Wilcoxon tests [32] compare each repair with plain U-Net; Holm correction covers six planned comparisons [6]. This follows recommendations to align validation with the application unit and report uncertainty rather than only a headline overlap metric [20, 21].

7.2 Integrity and segmentation results

The audited singleton-softmax module returns an all-one $B \times 1 \times 1 \times 1$ map, has zero attention variance and input response, changes its input by exactly zero, and gives exactly zero gradients for all score weights and biases. When shared U-Net weights are copied into plain and broken-gate networks, their logits are bitwise identical on a fixed input. This executable result agrees with the proposition and rules out training the broken branch as a distinct mechanism.

Table 2 shows small average improvements, but neither clean comparison is significant after correction: sigmoid $p = .238$ (Holm $p = .477$), integrity $p = .081$ (Holm $p = .324$). Under noise, integrity exceeds plain by .0140 (raw $p = .0287$, Holm $p = .1436$); under bias, by .0142 (raw $p = .0137$, Holm $p = .0821$). No comparison survives the prespecified family correction. A repaired gate is therefore demonstrably functional, but the present cohort does not establish a segmentation benefit.

7.3 Analysis-unit sensitivity

Treating the 539 slices as independent changes both effect estimates and inferential strength (Fig. 5). The clearest example is the sigmoid repair under noise: correct patient-level analysis gives mean

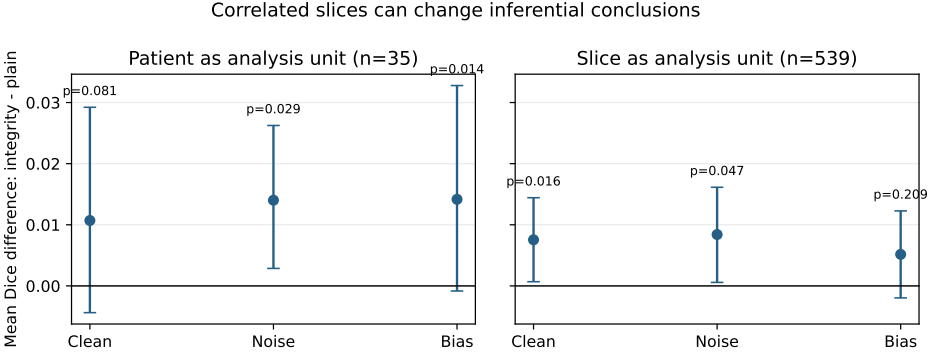


Fig. 5. Sensitivity analysis for the residual integrity gate. Patient inference uses 35 independent units; slice pseudoreplication uses 539 correlated observations and is shown only to quantify the methodological error. Error bars are percentile bootstrap intervals at the displayed unit; p labels are raw Wilcoxon values.

$\Delta = .0165$, raw $p = .115$, Holm $p = .344$; slice-level pseudoreplication gives $\Delta = .0147$, raw $p = .0011$, Holm $p = .0067$. The latter would appear significant while violating independence. We report it solely as a sensitivity analysis, not as evidence for the repair. The example shows how a silent software claim can be compounded by an invalid statistical unit.

8 Discussion

8.1 What the evidence establishes

GATECHECK establishes executable preconditions for attributing behavior to a learned component. Runtime success, plausible parameter counts, and end-to-end learning are insufficient. Value checks expose degeneracy; paired inputs expose responsiveness; forward comparison catches no-ops; backward probing catches graph disconnection; batch counterfactuals expose unintended sample interaction; and declared paired scenarios catch masks or options that silently disappear. A failing contract localizes the missing relation before expensive training, but passing contracts do not prove algorithmic correctness.

The case study separates mechanistic validity from task utility. Singleton softmax is invalid by proof and execution. Sigmoid and residual repairs pass their declared contracts, but neither produces a confirmatory clean Dice gain. This distinction prevents two errors: calling an inactive branch “attention,” and converting a software repair into an unsupported medical-performance claim.

8.2 Threats to validity

Construct validity. The 21 controlled mutation mechanisms are author-designed and partly repeated across spatial and sequence layouts. They span runtime, numeric, value, graph, and batch failures but are not a taxonomy. Controls are canonical small modules, not proof that every legitimate gate passes. A contract can also be wrongly specified: option influence is invalid when two documented options are semantically equivalent for a particular input.

Internal validity. The stress matrix and thresholds were fixed before the final expanded run, but mutation and probe design were developed together. This creates benchmark-author bias. The public-patch contracts were selected from developer descriptions before executing the outcomes, yet they were selected and reduced by the same researcher. No second annotator independently

verified intent. Threshold robustness, immutable hashes, and paired fixed controls reduce but do not remove this threat.

External validity. The controlled partitions cover spatial and sequence gates, while the parity check covers PyTorch and JAX. The public-patch pilot still contains only five reduced pairs from two Hugging Face repositories and executes in PyTorch on CPU. Full historical stacks, TensorFlow, accelerators, mixture-of-experts routing, quantization, distributed training, stochastic gates, and optimization-emergent failures remain untested. The pilot is a mechanism check, not a representative real-fault corpus; its exact interval makes the sample limitation explicit.

Case-study validity. The segmentation experiment uses one historical two-center dataset, 35 test patients, resized 2D inputs, a 31k-parameter model, and three seeds. There is no external cohort, 3D context, scanner subgroup analysis, or comparison with a tuned nnU-Net [15]. Corruptions are synthetic. Accordingly, this is evidence of the defect’s consequence and of correct analysis practice, not a state-of-the-art clinical model.

8.3 Adoption and remaining validation

GateCheck is intended for module unit tests and continuous integration. A developer declares applicable relations, runs generated stress cells, and keeps the fixed implementation as a regression control. The artifact includes a CI workflow and a protocol for future independent annotation. The next necessary evidence is a larger project-stratified corpus with complete historical environments, two contract annotators blinded to outcomes, and measurement of specification effort and false alarms in practice. Such evidence cannot be replaced by more executions of the current 96 controlled configurations. A medical-imaging performance paper would be a separate program requiring external cohorts and strong 3D baselines; it is not claimed here.

9 Reproducibility, Ethics, and Disclosure

The anonymous artifact contains source, seven automated tests, frozen protocols, dependency versions, all 17,380 execution rows, aggregate tables, threshold sweeps, case-study reanalysis, and vector figures. A single script reruns tests and analyses; a CI workflow is included. The real-fault manifest records project, pull request, full commit hash, date, contract, and claim boundary. Raw MRI data are not redistributed; the cited figshare release is CC BY 4.0. The study uses a public de-identified dataset and makes no clinical-deployment claim. No people were recruited and no participant data were collected. The generative-AI use that affected the research lifecycle is disclosed in the Methods.

10 Conclusion

A learned gate or attention interface should be credited only after evidence shows that the claimed relations are executable. GATECHECK turns local dynamics, forward/backward connectivity, batch isolation, and declared influence/invariance into millisecond-scale contracts. On 96 controlled configurations and 17,280 executions, the full suite separates all mutants from controls while shape checks miss most faults. PyTorch and JAX agree on matched definitions; five public-patch micro-replays connect the oracles to developer-confirmed mechanisms while making the small-sample boundary visible. The singleton-softmax case shows why integrity matters and why repair must not be confused with improved task accuracy. Passing contracts is evidence that a mechanism is active, not that a model is useful, safe, or universally correct.

11 Data Availability

The double-anonymous supplementary archive contains the source code, exact environment specification, frozen protocols, unit tests, all generated execution rows, aggregate results, figures, and paper source. Public patch provenance is provided through immutable commit URLs in the manifest. The Cheng MRI images are not redistributed; they remain available from the cited figshare record. To preserve review anonymity, the submission does not include an author-identifying repository URL. A public archival URL and explicit release license will be added after the anonymity period; until then the review archive is the authoritative artifact.

References

[1] James Bradbury, Roy Frostig, Peter Hawkins, et al. 2018. JAX: Composable Transformations of Python+NumPy Programs. <https://github.com/google/jax>.

[2] Jun Cheng. 2017. Brain Tumor Dataset. doi:10.6084/m9.figshare.1512427.v8 Version 8, CC BY 4.0.

[3] Jun Cheng, Wei Huang, Shuangliang Cao, et al. 2015. Enhanced Performance of Brain Tumor Classification via Tumor Region Augmentation and Partition. *PLOS ONE* 10, 10 (2015), e0140381. doi:10.1371/journal.pone.0140381

[4] Koen Claessen and John Hughes. 2000. QuickCheck: A Lightweight Tool for Random Testing of Haskell Programs. In *Proceedings of the Fifth ACM SIGPLAN International Conference on Functional Programming*. 268–279. doi:10.1145/351240.351266

[5] Bradley Efron and Robert J. Tibshirani. 1994. *An Introduction to the Bootstrap*. CRC Press.

[6] Sture Holm. 1979. A Simple Sequentially Rejective Multiple Test Procedure. *Scandinavian Journal of Statistics* 6, 2 (1979), 65–70.

[7] Jie Hu, Li Shen, Samuel Albanie, Gang Sun, and Enhua Wu. 2018. Squeeze-and-Excitation Networks. In *IEEE Conference on Computer Vision and Pattern Recognition*. 7132–7141. doi:10.1109/CVPR.2018.00745

[8] Hugging Face. 2022. OPT: Fix Softmax NaN in Half-Precision Mode. Commit d444edb3f63d. <https://github.com/huggingface/transformers/commit/d444edb3f63d0ab2e67150a15205f5024b93d7a8>

[9] Hugging Face. 2024. Fix Joint Attention Keyword Arguments Not Passed to FLUX Attention Processors. Commit acd6d2c42f0f. <https://github.com/huggingface/diffusers/commit/acd6d2c42f0f4f4fade262e8814279748a544b0ce>

[10] Hugging Face. 2026. Fix GPT-2 Attention Scaling Ignored in SDPA/FlashAttention. Commit 8757098be4d2. <https://github.com/huggingface/transformers/commit/8757098be4d2386d45900b30168855996ca22810>

[11] Hugging Face. 2026. GPT-2: Fix Encoder Attention Mask Being Silently Discarded in Cross-Attention. Commit e0aeab4c9f71. <https://github.com/huggingface/transformers/commit/e0aeab4c9f7102d18d15b478ecec075f9e1658223>

[12] Hugging Face. 2026. MusicGen Melody: Fix Conditioning Silently Dropped at Generation Step Zero. Commit 5bed62964cfe. <https://github.com/huggingface/transformers/commit/5bed62964cfe54a806a620eddcf459b29fe4731>

[13] Nargiz Humbatova, Gunel Jahangirova, Gabriele Bavota, Vincenzo Riccio, Andrea Stocco, and Paolo Tonella. 2020. Taxonomy of Real Faults in Deep Learning Systems. In *Proceedings of the 42nd International Conference on Software Engineering*. 1110–1121. doi:10.1145/3377811.3380395

[14] Nargiz Humbatova, Gunel Jahangirova, and Paolo Tonella. 2021. DeepCrime: Mutation Testing of Deep Learning Systems Based on Real Faults. In *Proceedings of the 30th ACM SIGSOFT International Symposium on Software Testing and Analysis*.

[15] Fabian Isensee, Paul F. Jaeger, Simon A. A. Kohl, Jens Petersen, and Klaus H. Maier-Hein. 2021. nnU-Net: A Self-configuring Method for Deep Learning-based Biomedical Image Segmentation. *Nature Methods* 18 (2021), 203–211. doi:10.1038/s41592-020-01008-z

[16] Gunel Jahangirova, Nargiz Humbatova, Jinhan Kim, Shin Yoo, and Paolo Tonella. 2024. Real Faults in Deep Learning Fault Benchmarks: How Real Are They? *arXiv preprint arXiv:2412.16336* (2024). doi:10.48550/arXiv.2412.16336

[17] Ho Young Jhoo, Sehoon Kim, Woosung Song, Kyuyeon Park, DongKwon Lee, and Kwangkeun Yi. 2021. A Static Analyzer for Detecting Tensor Shape Errors in Deep Neural Network Training Code. *arXiv preprint arXiv:2112.09037* (2021). doi:10.48550/arXiv.2112.09037

[18] Jiawei Liu, Jinkun Lin, Fabian Ruffy, Cheng Tan, Jinyang Li, Aurojit Panda, and Lingming Zhang. 2023. NNSmith: Generating Diverse and Valid Test Cases for Deep Learning Compilers. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*. 530–543. doi:10.1145/3575693.3575707

[19] Lei Ma, Fuyuan Zhang, Jiyuan Sun, Minhui Xue, Bo Li, Felix Juefei-Xu, Chao Xie, Li Li, Yang Liu, Jianjun Zhao, and Yadong Wang. 2018. DeepMutation: Mutation Testing of Deep Learning Systems. *arXiv preprint arXiv:1805.05206* (2018). doi:10.48550/arXiv.1805.05206

- [20] Lena Maier-Hein, Annika Reinke, Patrick Godau, et al. 2024. Metrics Reloaded: Recommendations for Image Analysis Validation. *Nature Methods* 21 (2024), 195–212. doi:10.1038/s41592-023-02151-z
- [21] John Mongan, Linda Moy, and Charles E. Kahn. 2020. Checklist for Artificial Intelligence in Medical Imaging (CLAIM): A Guide for Authors and Reviewers. *Radiology: Artificial Intelligence* 2, 2 (2020), e200029. doi:10.1148/ryai.2020200029
- [22] Mohammad Mehdi Morovati, Amin Nikanjam, Foutse Khomh, and Zhen Ming Jiang. 2023. Bugs in Machine Learning-Based Systems: A Faultload Benchmark. *Empirical Software Engineering* 28, 3 (2023), 62. doi:10.1007/s10664-023-10291-1
- [23] Augustus Odena and Ian Goodfellow. 2019. TensorFuzz: Debugging Neural Networks with Coverage-Guided Fuzzing. *Proceedings of Machine Learning Research* 97 (2019), 4901–4911.
- [24] Ozan Oktay, Jo Schlemper, Loic Le Folgoc, et al. 2018. Attention U-Net: Learning Where to Look for the Pancreas. *arXiv preprint arXiv:1804.03999* (2018). doi:10.48550/arXiv.1804.03999
- [25] Adam Paszke, Sam Gross, Francisco Massa, et al. 2019. PyTorch: An Imperative Style, High-Performance Deep Learning Library. In *Advances in Neural Information Processing Systems*, Vol. 32.
- [26] Kexin Pei, Yinzhi Cao, Junfeng Yang, and Suman Jana. 2017. DeepXplore: Automated Whitebox Testing of Deep Learning Systems. In *Proceedings of the 26th Symposium on Operating Systems Principles*. 1–18. doi:10.1145/3132747.3132785
- [27] Hung Viet Pham, Thibaud Lutellier, Weizhen Qi, and Lin Tan. 2019. CRADLE: Cross-Backend Validation to Detect and Localize Bugs in Deep Learning Libraries. In *Proceedings of the 41st International Conference on Software Engineering*. 1027–1038. doi:10.1109/ICSE.2019.00107
- [28] Mohammad Rezaalipour and Carlo A. Furia. 2023. An Annotation-based Approach for Finding Bugs in Neural Network Programs. *Journal of Systems and Software* 201 (2023), 111669. doi:10.1016/j.jss.2023.111669
- [29] Olaf Ronneberger, Philipp Fischer, and Thomas Brox. 2015. U-Net: Convolutional Networks for Biomedical Image Segmentation. In *Medical Image Computing and Computer-Assisted Intervention*. 234–241. doi:10.1007/978-3-319-24574-4_28
- [30] Arnab Sharma, Caglar Demir, Axel-Cyrille Ngonga Ngomo, and Heike Wehrheim. 2021. MLCheck: Property-Driven Testing of Machine Learning Models. In *20th IEEE International Conference on Machine Learning and Applications*. 738–745. doi:10.1109/ICMLA52953.2021.00121
- [31] Florian Tambon, Amin Nikanjam, Le An, Foutse Khomh, and Giuliano Antoniol. 2023. Silent Bugs in Deep Learning Frameworks: An Empirical Study of Keras and TensorFlow. *arXiv preprint arXiv:2112.13314* (2023). doi:10.48550/arXiv.2112.13314
- [32] Frank Wilcoxon. 1945. Individual Comparisons by Ranking Methods. *Biometrics Bulletin* 1, 6 (1945), 80–83. doi:10.2307/3001968
- [33] Sanghyun Woo, Jongchan Park, Joon-Young Lee, and In So Kweon. 2018. CBAM: Convolutional Block Attention Module. In *European Conference on Computer Vision*. 3–19. doi:10.1007/978-3-030-01234-2_1
- [34] Ming Yan, Junjie Chen, Xiangyu Zhang, Lin Tan, Gan Wang, and Zan Wang. 2021. Exposing Numerical Bugs in Deep Learning via Gradient Back-Propagation. In *Proceedings of the 29th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. 627–638. doi:10.1145/3468264.3468612
- [35] Yuhao Zhang, Luyao Ren, Liqian Chen, Yingfei Xiong, Shing-Chi Cheung, and Tao Xie. 2020. Detecting Numerical Bugs in Neural Network Architectures. In *Proceedings of the 28th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. 826–837. doi:10.1145/3368089.3409720

A Reproducibility details

Table 3. Cumulative probe-family ablation at configuration level.

Strategy	Mutant recall	Control specificity	False-positive rate
Runtime/shape only	.095	1.000	.000
+ finiteness	.190	1.000	.000
+ value dynamics	.524	1.000	.000
+ forward/backward	.810	1.000	.000
Full GATECHECK	1.000	1.000	.000

Commands. From the artifact root, run
bash a_star_research/run_all.sh

Machine-readable outputs. The spatial and sequence execution CSVs contain case, stress factors, raw statistics, Boolean flags, runtime, and correctness. Separate files hold public-patch and cross-framework executions, immutable fault provenance, configuration aggregates, ablations, threshold sensitivity, and patient/slice sensitivity. Summary JSON files contain thresholds, counts, intervals, framework versions, and timing.